

In the above example, the `$check` variable has been used by applying the ternary operator. By using the inbuilt `isset()` function, you can check if the `$HTTP_POST_VARS` array comprises elements.

Basically, both GET and POST methods are used to send data to the data processing page. Though both methods are used in form data handling, there are differences in their working method. Some remarkable differences are:

- The data remains visible in the address bar since contents are passed as part of the URL and as a query string in the GET method. In the POST method, data is not visible as contents are passed to the script as an input file.
- Using the GET method, you can insert a bookmark link whereas a page link can never be bookmarked with the POST method.
- You can only transfer 1KB of data through the GET method. Large amounts of data can be transferred through POST method, which is determined by the 'post_max_size' directive in php.ini.
- In the GET method, data is submitted as a part of a URL, while in the POST method data is submitted as part of an http request.
- In the GET method, data is swift but not secure. On the other hand, POST data is secure and slow as compared to GET.

7.6. File upload

In the sections above, we have seen simple form input. Now we will discuss about the features that PHP creates to work with inputs. Here, we will learn about file uploading in PHP. PHP makes it possible to upload files to the server. To begin with, we should first create HTML forms including file upload fields and an ENCTYPE argument:

```
ENCTYPE="multipart/form-data"
```

Look at the HTML form below, used for uploading files:

```
<html>
<body><form action="upload1.php" method="post"
enctype="multipart/form-data">
<label for="file">Filename:</label>
<input type="file" name="uploadedfile" id="file"
/>
<br />
<input type="submit" name="submit" value="Upload"
```